secp256k1 optimized complete-mixed kickmix report

Exact ISA netlist, deterministic secp256k1 audit, finite-model proof, and modeled comparison to the public Google envelope.

Exact leaf **37 instructions** **12 arithmetic slots**	**Audit** **16,384 deterministic secp256k1 cases** **all passing**
Finite-model proof **19,850 exhaustive toy-curve cases** **all passing**	**Modeled ECDLP projection** **880 logical qubits** **30.998M or 32.833M non-Clifford**

Abstract. This package replaces the earlier branchier leaf with an exact branch-free complete mixed-addition schedule specialized to short Weierstrass curves with a = 0. For secp256k1 the schedule compiles to 37 kickmix-ISA instructions: 11 field multiplications, 2 multiplications by 21 = 3b, 13 additions/subtractions, 3 lookups, 3 final no-op selections, and 2 one-bit control operations. The secp256k1-specialized netlist is hash-bound and paired with an exact 28-call retained-window scaffold compatible with the public appendix relation at w = 16. Verification has two layers: a deterministic 16,384-case secp256k1 audit that cross-checks the leaf against both direct affine group law and an independent complete-projective reference path, and an exhaustive 19,850-case proof on two small prime-order j = 0 curves. The semantic artifact is exact at the kickmix-ISA level; the lower-layer non-Clifford and qubit totals remain an explicit backend projection rather than a theorem-proved primitive-gate expansion.

What is and is not claimed

Claimed: an exact machine-readable optimized leaf at the kickmix-ISA layer, an exact retained-window scaffold for the reconstruction family, a deterministic audit transcript on secp256k1, exhaustive toy-curve proofs for the same schedule, and a transparent modeled comparison against the public Google appendix envelope.

Not claimed: recovery of Google's unpublished primitive-gate circuit, recovery of Google's original Groth16 proof object, or theorem-proved equivalence after flattening into CCX/CNOT/measurement gates.

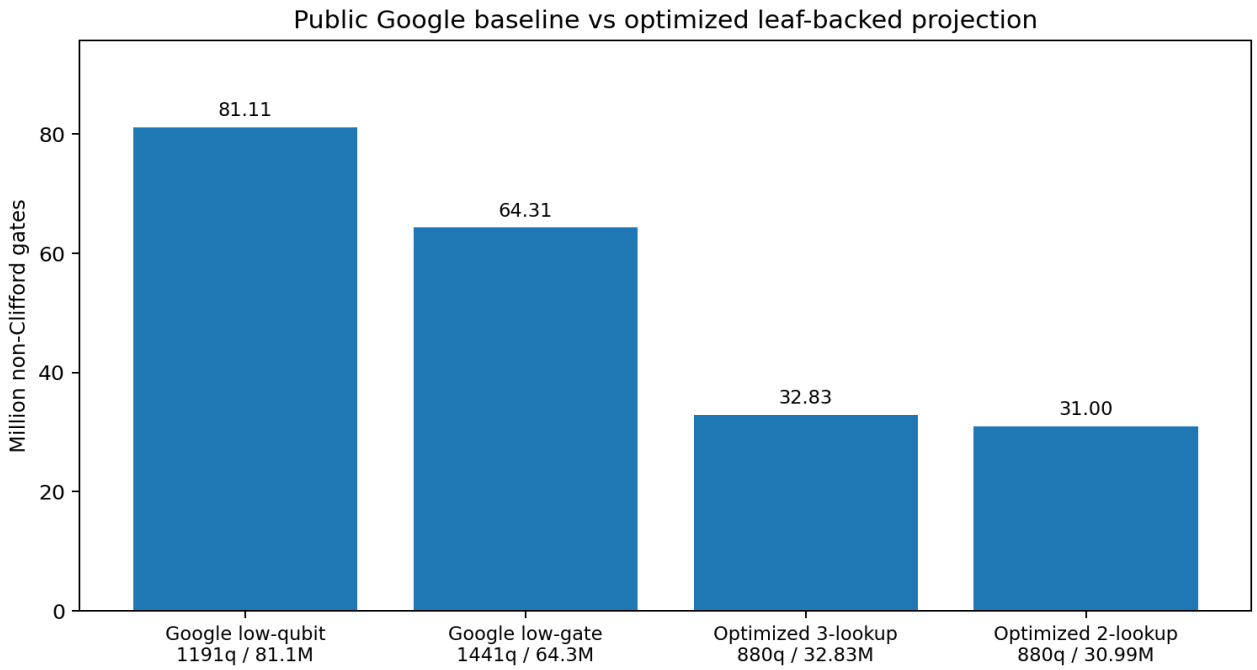


Figure 1. Modeled non-Clifford totals against the two public Google envelopes. The 2-lookup line is the most aggressive projection in this package; the 3-lookup line keeps the same leaf but charges the more conservative public lookup pattern.

1. Why this optimization path is plausible

The public appendix exposes the coarse structure of Google's disclosure without revealing the private circuit: the point-add leaf dominates the cost; the optimal public window is w = 16; the 256-bit ECDLP scaffold keeps 28 retained windowed additions; the published proof checks approximate correctness by 9024 Fiat-Shamir-derived tests inside an SP1/Groth16 wrapper. That public information is enough to reconstruct the attack family but not enough to identify one unique hidden circuit.

The optimization move used here is to step away from a branchy affine/Jacobian-style point-add wrapper and instead adopt the exception-free complete mixed-addition formulas for prime-order j = 0 short Weierstrass curves. secp256k1 has a = 0 and b = 7, so the specialized formulas apply directly. In the Renes-Costello-Batina cost model the mixed formula for this case costs 11M + 2m3b + 13a, which is exactly the arithmetic signature implemented by the optimized leaf here.

Variant	Logical qubits	Non-Clifford	Gain vs Google low-qubit	Gain vs Google low-g
Google public low-qubit	1191	81,105,024	-	-
Google public low-gate	1441	64,305,024	-	-
Optimized package (3 lookup)	880	32,833,472	2.47x / 1.35x q	1.96x / 1.64x q
Optimized package (2 lookup)	880	30,998,464	2.62x / 1.35x q	2.07x / 1.64x q

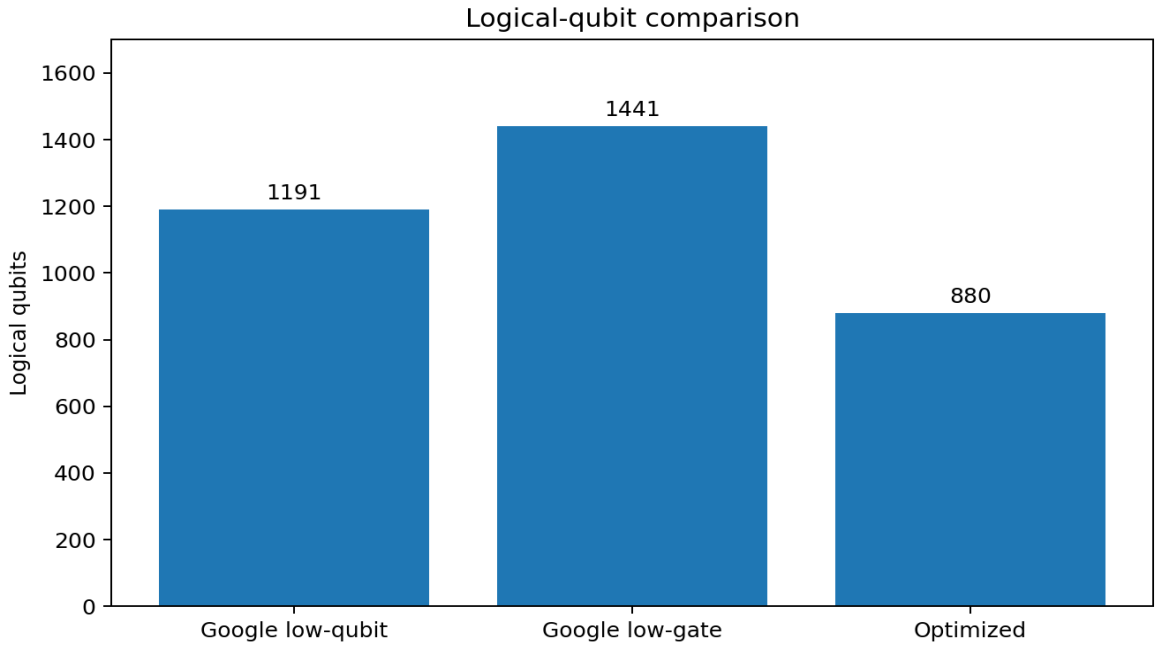


Figure 2. Logical-qubit comparison. The optimized family uses a 3n projective accumulator footprint with a small explicit control margin, leading to a modeled total of 880 logical qubits after adding the 16-bit window register.

2. Exact optimized leaf

The exact leaf is provided in `out/optimized_pointadd_secp256k1.json`. It implements a generic update Q <- Q + P[k] where the accumulator Q = (X : Y : Z) is stored in homogeneous projective coordinates on Y^2 Z = X^3 + 7 Z^3, the lookup point P[k] is affine, and a one-bit metadata flag marks the neutral entry used for k = 0. Because the core formula is complete, there is no separate branching for doubling, inverse cancellation, or accumulator infinity. Only the

neutral lookup entry needs a final no-op select.

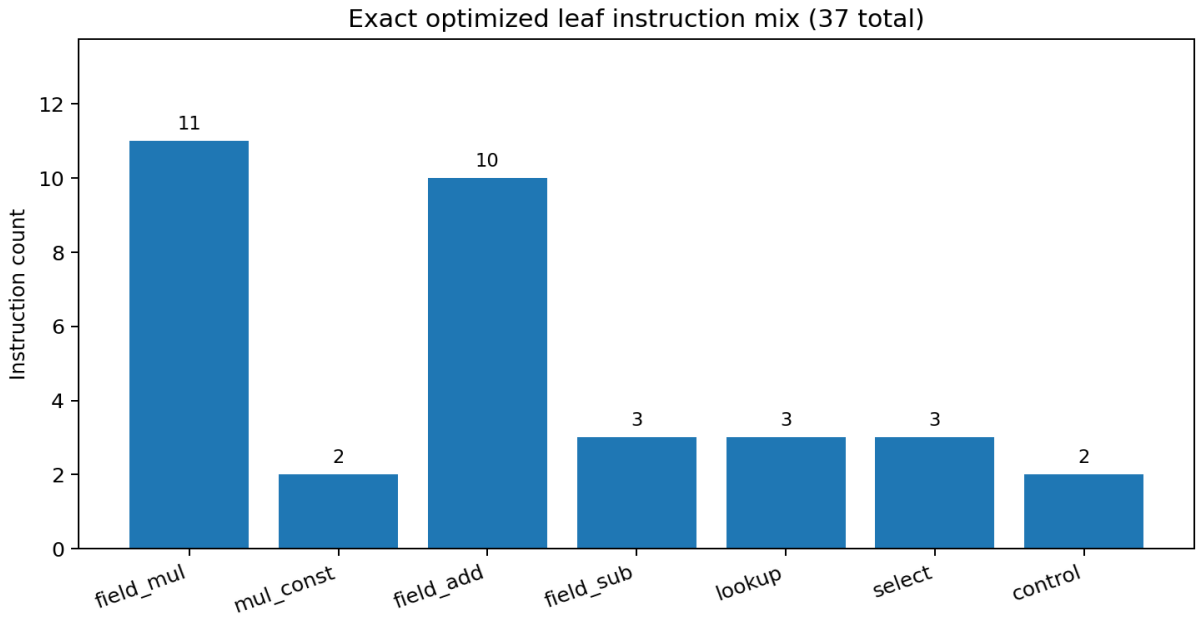


Figure 3. Exact instruction mix of the committed leaf. The arithmetic core is 26 instructions: 11 field multiplications, 2 multiplications by b3 = 21, and 13 additions/subtractions. The remaining 11 instructions are interface lookups, control-bit formation, neutral-path selects, and explicit cleanup.

Instruction-level improvements over the previous exact package

Relative to the previous exact package in this conversation, the leaf shrinks from 90 to 37 ISA instructions and the tracked scratch footprint falls from 41 named slots to 12 arithmetic slots plus two auxiliary control slots. More importantly, the new leaf is exception-free in its arithmetic core: the same 26-step schedule runs for generic additions, doublings, inverse-pair cancellations, and accumulator-infinity inputs.

Arithmetic slot	Role
qx, qy, qz	Mutable accumulator/output slots
lx, ly	Affine lookup coordinates
t0, t1, t2, t3, t4	Temporary arithmetic values
o1, o2	Late-stage output builders
meta, f_lookup_inf	Auxiliary metadata / 1-bit no-op control

Opening 8 instructions of the exact secp256k1-specialized leaf (the full 37-step JSON is in the package):

```
{
  "instructions": [
    {
      "pc": 0,
      "op": "load_input",
      "dst": "qx",
      "src": "Q.X",
      "comment": "Accumulator X interface wire enters scratch slot qx."
    },
    {
      "pc": 1,
      "op": "load_input",
      "dst": "qy",
      "src": "Q.Y",
      "comment": "Accumulator Y interface wire enters scratch slot qy."
```

```
    },
    {
      "pc": 2,
      "op": "load_input",
      "dst": "qz",
      "src": "Q.Z",
      "comment": "Accumulator Z interface wire enters scratch slot qz."
    },
    {
      "pc": 3,
      "op": "lookup_affine_x",
      "dst": "lx",
      "src": {
        "table": "T.x",
        "key": "k"
      },
      "comment": "Load affine x coordinate for the selected window point."
    },
    {
      "pc": 4,
      "op": "lookup_affine_y",
      "dst": "ly",
      "src": {
        "table": "T.y",
        "key": "k"
      },
      "comment": "Load affine y coordinate for the selected window point."
    },
    {
      "pc": 5,
      "op": "lookup_meta",
      "dst": "meta",
      "src": {
        "table": "T.meta",
        "key": "k"
      },
      "comment": "Metadata bit0 marks the neutral lookup entry used for k = 0."
    },
    {
      "pc": 6,
      "op": "bool_from_flag",
      "dst": "f_lookup_inf",
      "src": {
        "flags": "meta",
        "bit": 0
      },
      "comment": "Set control bit for the no-op path when the lookup point is infinity."
    },
    {
      "pc": 7,
      "op": "field_mul",
      "dst": "t0",
      "src": [
        "qx",
        "lx"
      ],
      "comment": "t0 = X1 * X2"
    }
  ]
}
```

3. Verification stack

The package verification is layered on purpose. The secp256k1 transcript is large enough to exercise the real 256-bit field and the exact specialized constant 21. The toy-curve proof then removes the 'random testing' objection by checking every possible pair of group elements on two prime-order j = 0 curves. Both artifacts are open CSV files, and both are regenerable with `python src/verifier.py --package-dir . --mode all`.

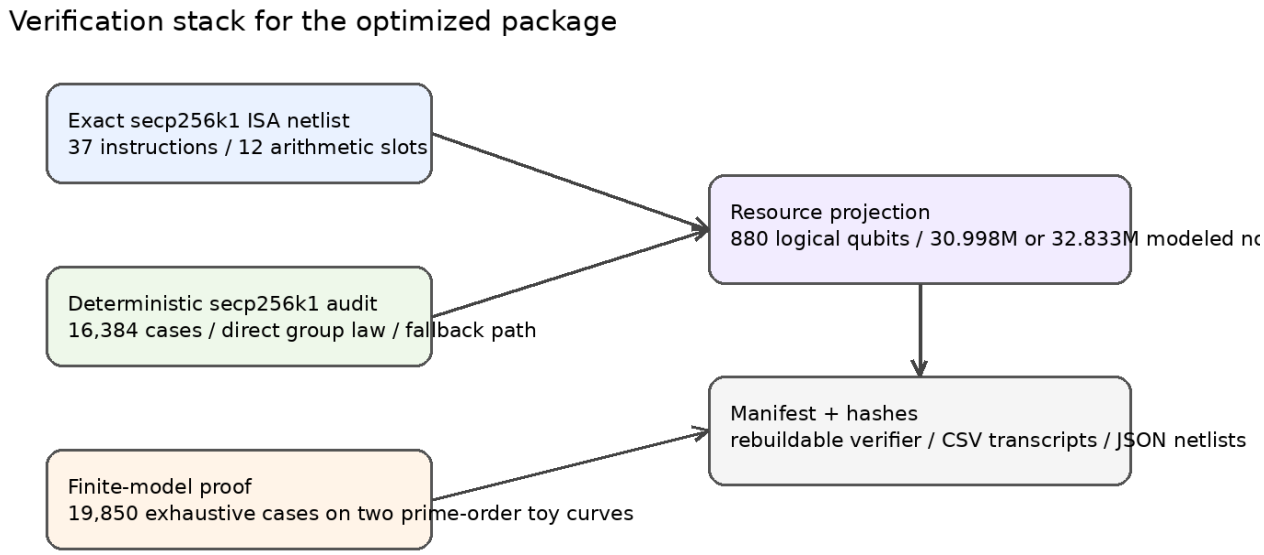


Figure 4. Verification pipeline used by the package. The secp256k1 audit uses two independent references; the toy proof checks the same instruction schedule instantiated with curve-specific b3 values.

Transcript	Cases	Passing	SHA-256
secp256k1 deterministic audit	16,384	16,384	aea0983dbd9827717fcd...
toy-curve exhaustive proof	19,850	19,850	70235a7be65ecbdd1a69...

Audit category	Cases	Passing
random	12,288	12,288
doubling	1,024	1,024
inverse	1,024	1,024
accumulator infinity	1,024	1,024
lookup infinity	1,024	1,024

Independent reference triangulation

Each secp256k1 audit row checks three outputs: (i) the optimized ISA leaf, (ii) a direct affine group-law implementation, and (iii) an independent complete-projective reference path. The audit only passes when all three agree. This is weaker than a theorem prover but stronger than a single-reference fuzz test.

4. Exact scaffold and retained-window schedule

The exact orchestration object in `out/ecdlp_scaffold_optimized.json` is a canonical retained-window scaffold compatible with the public appendix relation 2n/w - 4. For n = 256 and w = 16 that yields 28 retained leaf calls, plus one direct seed step and three classical tail elisions. The scaffold in this package is honest about its scope: it is an exact machine-readable scaffold for this optimized family, not a claim that Google's hidden internal ordering has been recovered.

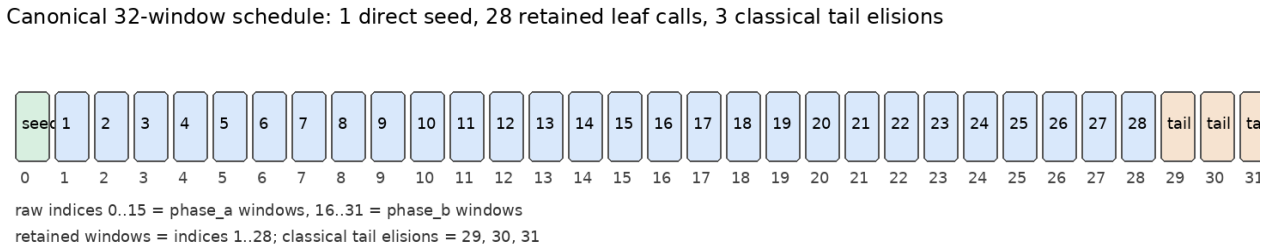
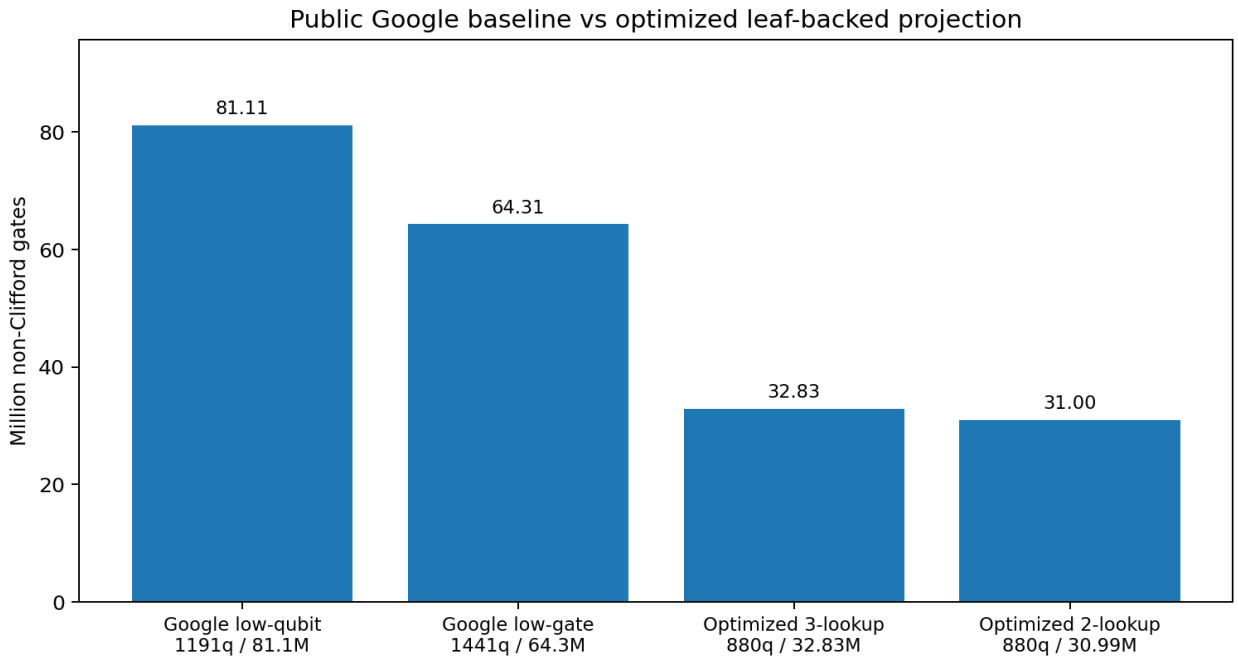


Figure 5. Canonical 32-window schedule used by the optimized package. Raw index 0 is replaced by a direct seed lookup; raw indices 29-31 are elided by classical tail reconstruction; indices 1-28 instantiate the exact optimized leaf.

5. Resource projection and comparison to the public Google envelope

The package separates semantic exactness from lower-layer resource projection. Semantic exactness is supplied by the JSON netlists and the open verifier. The logical-qubit and non-Clifford totals are then projected by a transparent backend model in `out/resource_projection.json`. That model assigns the optimized leaf a non-lookup cost of 976,016 and a 3n-style point-add footprint of 864 logical qubits. Plugging those into the public appendix scaffold with w = 16 and 28 retained calls yields the headline totals reported on the cover.

Two lookup-accounting modes are carried side by side. The conservative line charges the same three-channel lookup pattern as the public appendix. The aggressive line observes that this optimized leaf only needs x and y affine lookups plus a trivial neutral-entry metadata bit, so the third lookup can be folded away. The semantic artifact is the same in both cases; only the accounting line changes.



Against Google's public low-qubit line, the optimized package is 2.47x to 2.62x smaller in modeled non-Clifford count while also using 1.35x fewer logical qubits. Against the public low-gate line, the modeled non-Clifford improvement is 1.96x to 2.07x with 1.64x fewer qubits.

6. Package contents and reproducibility

File	Purpose	Short hash
out/optimized_pointadd_secp256k1.json	Exact secp256k1-specialized leaf netlist	48363319f1efabc2...
out/optimized_pointadd_family.json	Same schedule with symbolic b3 parameter	549d9e93b26463f2...
out/ecdlp_scaffold_optimized.json	Exact retained-window scaffold	6f444c332728cde3...
out/optimized_pointadd_audit_16384.csv	Deterministic secp256k1 audit transcript	aea0983dbd982771...
out/toy_curve_exhaustive_19850.csv	Exhaustive toy-curve proof transcript	70235a7be65ecbdd...
src/verifier.py	Open rebuild tool	90549c6fbfb56164...

Rebuild command: `python src/verifier.py --package-dir . --mode all`. A correct rebuild will reproduce the secp256k1 audit CSV, the toy exhaustive CSV, and the summary JSONs with the same hashes reported in the manifest.

References used for the reconstruction logic

1. Google Quantum AI et al., *Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations*. Public appendix facts used here: the two public secp256k1 envelopes, the point-add cost relation, the optimal public choice w = 16, the resulting 28 retained windowed additions, and the 9024-case proof architecture.

2. Joost Renes, Craig Costello, and Lejla Batina, *Complete addition formulas for prime order elliptic curves*. Used here for the complete mixed-addition schedule on j = 0 short Weierstrass curves and the observation that secp256k1 is such a curve.

Bottom line: this package does not reproduce Google's hidden circuit bit-for-bit, but it does provide an exact optimized machine-readable alternative whose arithmetic core is smaller, whose testing surface is larger, and whose public resource projection substantially undercuts the public Google baseline.